HILL CLIMB SEARCHING FOR 4-QUEENS

```
import random

def heuristic(state):
    attacks = 0
    n = len(state)

    for i in range(n):
        for j in range(i + 1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                attacks += 1

    return attacks


def get_best_neighbor(state):
    n = len(state)
    best_h = heuristic(state)
    best_neighbors = []

    for col in range(n):
        for row in range(n):
            if row == state[col]:
                continue

            neighbor = state.copy()
            neighbor[col] = row
            h = heuristic(neighbor)

            if h < best_h:
                best_h = h
                best_neighbors = [neighbor]
            elif h == best_h and h < heuristic(state):
                best_neighbors.append(neighbor)

    if best_neighbors:
        return random.choice(best_neighbors), best_h

    return state, heuristic(state)


def hill_climbing(initial_state):
    n = len(initial_state)
    current = initial_state.copy()
    restarts = 0
    max_restarts = 1000

    while heuristic(current) != 0:
        neighbor, h = get_best_neighbor(current)

        if h < heuristic(current):
            current = neighbor
            print("State:", current, "Conflicts:", h)
        else:
            restarts += 1

            if restarts > max_restarts:
                print("Maximum restarts reached.")
                return None

            current = [random.randint(0, n - 1) for _ in range(n)]
            print("Random Restart:", current)

    print("Solution found!")
    print("Total restarts:", restarts)
    return current


def print_board(state):
    for row in range(len(state)):
        print(" ".join("Q" if state[col] == row else "."
                       for col in range(len(state))))
```

```
if __name__ == "__main__":
    try:
        state = list(map(int, input("Enter 4 row values (0-3): ").split()))

        if len(state) != 4 or any(x < 0 or x > 3 for x in state):
            raise ValueError

        solution = hill_climbing(state)

        if solution:
            print("\nFinal Solution:", solution)
            print_board(solution)

    except ValueError:
        print("Invalid input! Enter exactly 4 integers between 0 and 3.")
```

```
Enter 4 row values (0-3): 3 1 2 0
Random Restart: [0, 3, 1, 2]
Random Restart: [0, 0, 1, 1]
State: [0, 0, 3, 1] Conflicts: 1
State: [2, 0, 3, 1] Conflicts: 0
Solution found!
Total restarts: 2

Final Solution: [2, 0, 3, 1]
. Q . .
. . . Q
Q . . .
. . Q .
```